

Gateway Scaffold

Gateway Scaffold (Event-Driven, Redis-Backed, Polyglot)

Overview

This document provides a complete **Gateway scaffold** for your event-driven, Redis-backed, polyglot architecture. The Gateway acts as the central orchestrator between:

- Frontend (HTTP + SSE)
- Redis (Pub/Sub)
- Agent Runtime Services (Python + Node)
- Database (MongoDB/Postgres)

It supports human-in-the-loop workflows, task/state management, and normalized event streaming to the frontend.

Folder Structure

```
/gateway
  server.ts
  /routes
    tasks.ts
    sessions.ts
    approvals.ts
  /sse
    eventStream.ts
  /redis
    pub.ts
    sub.ts
  /db
    TaskModel.ts
    SessionModel.ts
  /events
    normalizeEvent.ts
    eventTypes.ts
  /utils
    logger.ts
```

File: server.ts

```
import Fastify from "fastify";
import cors from "@fastify/cors";
import tasksRouter from "../routes/tasks";
import sessionsRouter from "../routes/sessions";
import approvalsRouter from "../routes/approvals";
import { initRedisSubscriber } from "../redis/sub";

const app = Fastify({ logger: true });

app.register(cors, { origin: "*" });

app.register(tasksRouter, { prefix: "/tasks" });
app.register(sessionsRouter, { prefix: "/sessions" });
app.register(approvalsRouter, { prefix: "/approvals" });

initRedisSubscriber(app);

app.listen({ port: 3001 }, () => {
  console.log("Gateway running on port 3001");
});
```

File: /routes/tasks.ts

```
import { FastifyInstance } from "fastify";
import { TaskModel } from "../db/TaskModel";
import { redisPub } from "../redis/pub";

export default async function tasksRouter(app: FastifyInstance) {
  app.post("/", async (req, res) => {
    const { agentId, input } = req.body as { agentId: string; input: any };

    const task = await TaskModel.create({
      agentId,
      input,
      status: "PENDING",
      createdAt: new Date()
    });

    await redisPub.publish(
      "tasks",
      JSON.stringify({ type: "task.created", taskId: task.id })
    );

    res.send({ taskId: task.id });
  });
}
```

File: /routes/sessions.ts

```
import { FastifyInstance } from "fastify";
import { createEventStream } from "../sse/eventStream";

export default async function sessionsRouter(app: FastifyInstance) {
  app.get("/:sessionId/events", async (req, res) => {
    const { sessionId } = req.params as { sessionId: string };
    createEventStream(sessionId, res);
  });
}
```

File: /routes/approvals.ts

```
import { FastifyInstance } from "fastify";
import { TaskModel } from "../db/TaskModel";
import { redisPub } from "../redis/pub";

export default async function approvalsRouter(app: FastifyInstance) {
  app.post("/:taskId", async (req, res) => {
    const { taskId } = req.params as { taskId: string };
    const { choice } = req.body as { choice: string };

    await TaskModel.update(taskId, {
      status: "QUEUED",
      "humanInteraction.selectedChoice": choice,
      updatedAt: new Date()
    });

    await redisPub.publish(
      "tasks",
      JSON.stringify({ type: "task.resume", taskId })
    );

    res.send({ ok: true });
  });
}
```

File: /sse/eventStream.ts

```
import { redisSub } from "../redis/sub";

export function createEventStream(sessionId: string, res: any) {
  res.raw.setHeader("Content-Type", "text/event-stream");
  res.raw.setHeader("Cache-Control", "no-cache");
  res.raw.setHeader("Connection", "keep-alive");

  const channel = `session:${sessionId}:events`;

  const sub = redisSub.duplicate();
  sub.subscribe(channel);

  sub.on("message", (_, message) => {
    res.raw.write(`data: ${message}\n\n`);
  });

  res.raw.on("close", () => {
    sub.unsubscribe(channel);
    sub.quit();
  });
}
```

File: /redis/pub.ts

```
import Redis from "ioredis";

export const redisPub = new Redis({
  host: "localhost",
  port: 6379
});
```

File: /redis/sub.ts

```
import Redis from "ioredis";
import { normalizeEvent } from "../events/normalizeEvent";

export const redisSub = new Redis({
  host: "localhost",
  port: 6379
});

export function initRedisSubscriber(app: any) {
  redisSub.subscribe("agent.logs");
  redisSub.subscribe("agent.events");

  redisSub.on("message", (channel, message) => {
    const raw = JSON.parse(message);
    const event = normalizeEvent(channel, raw);
    app.log.info({ event }, "Received agent event");
    // NOTE: In a full implementation, you would also fan-out
    // to per-session Redis channels like `session:${event.sessionId}:events`
  });
}
```

File: /events/normalizeEvent.ts

```
export function normalizeEvent(channel: string, raw: any) {
  return {
    sessionId: raw.sessionId,
    type: raw.type,
    payload: raw.payload,
    timestamp: Date.now(),
    source: channel
  };
}
```

File: /events/eventTypes.ts

```
export type GatewayEventType =  
  | "task.created"  
  | "task.resume"  
  | "task.updated"  
  | "approval.requested"  
  | "log.append"  
  | "output.partial"  
  | "output.final"  
  | "session.stateChanged";
```

File: /db/TaskModel.ts

```
// Placeholder DB model. Replace with real MongoDB/Postgres implementation.  
  
export const TaskModel = {  
  async create(data: any) {  
    // TODO: Insert into DB  
    return { id: crypto.randomUUID(), ...data };  
  },  
  
  async update(id: string, data: any) {  
    // TODO: Update DB record  
    return true;  
  }  
};
```

File: /db/SessionModel.ts

```
export const SessionModel = {  
  async create(data: any) {  
    // TODO: Insert into DB  
    return { id: crypto.randomUUID(), ...data };  
  }  
};
```

File: /utils/logger.ts

```
export function logInfo(message: string, meta?: any) {
  console.log(message, meta || "");
}

export function logError(message: string, meta?: any) {
  console.error(message, meta || "");
}
```

How This Relates to Your Existing Backend Scaffold

Your previously generated **backend scaffold** is still valid, but it now plays the role of an **Agent Runtime Service**:

- It subscribes to Redis (tasks, task.resume, etc.)
- It runs agents (LLM, tools, workflows)
- It publishes events back to Redis (agent.events, agent.logs, session:{id}:events)

The **Gateway** sits in front of it and:

- Exposes HTTP + SSE to the frontend
- Manages task/session state
- Bridges Redis  Frontend

Event flow:

```
Frontend (HTTP + SSE)
  ↓
Gateway (Fastify)
  ↓ ↑
Redis Pub/Sub
  ↑ ↓
Agent Runtime Services (Python + Node)
```

Summary

This Gateway scaffold gives you:

- A Fastify-based API Gateway
- SSE event streaming for sessions
- Redis Pub/Sub integration
- Task and session routing
- Human-in-the-loop approval handling

- A clean separation between Gateway and Agent Runtime Services

It is fully aligned with your event-driven, polyglot architecture and ready to be dropped into your repo as a starting point.